

CONTROL DE VERSIONES CON GIT Y GITHUB

VERSION CONTROL SYSTEM Y TRABAJO EN EQUIPO

Los sistemas de control de versiones (o VCS, por sus siglas en inglés: Version Control System) son herramientas de software diseñadas para gestionar y registrar los cambios realizados en un conjunto de archivos, como el código fuente de un proyecto.

Gracias a los VCS, los equipos de desarrollo pueden colaborar de forma rápida y eficiente mientras preservan la integridad del código. Esto permite a los programadores trabajar simultáneamente con la tranquilidad de que los conflictos de edición se gestionan de manera segura.

Al operar a nivel del sistema de archivos, un VCS rastrea continuamente la adición, modificación o eliminación de archivos y directorios. Para mantener la integridad del proyecto, las actualizaciones individuales se guardan periódicamente en registros de cambios llamados confirmaciones (commits). Posteriormente, estos cambios se fusionan de manera segura con el código fuente principal, garantizando un historial completo, una copia de seguridad constante y una gestión de conflictos transparente para todo el equipo.

En la actualidad, los VCS son indispensables para mantener flujos de trabajo eficientes, ya que constituyen la única “fuente de verdad” para el equipo. Actúan como una red de seguridad vital, evitan que la pérdida accidental o la modificación incorrecta de un archivo local ponga en riesgo todo el proyecto y permiten que cada desarrollador contribuya a la base de código de forma independiente y segura.

GIT Y GITHUB

Git y GitHub no son lo mismo. La diferencia principal es que Git es un sistema de control de versiones que se ejecuta de forma local para rastrear los cambios en archivos, mientras que GitHub es una plataforma en la nube que aloja esos proyectos de Git para facilitar el respaldo y el trabajo colaborativo.

Por un lado, Git nació para tomar el espacio de otros sistemas de control, siendo su principal creador Linus Torvalds (el mismo creador de Linux). A diferencia de la mayoría de los VCS tradicionales, Git almacena cada versión guardada como una ‘instantánea’ en lugar de una lista de cambios archivo por archivo. Esto te permite trabajar en tu propia copia local, registrar el historial, consultar instantáneas antiguas cuando lo necesites y crear nuevas cada vez que modificas el proyecto.

Por otro lado, GitHub se basa en Git alojando esos proyectos (denominados repositorios) en la nube. Además, añade herramientas visuales, de planeación y de revisión de código que facilitan a los equipos combinar su trabajo y colaborar de forma segura desde cualquier lugar.

DEFINICIONES CON EJEMPLO

1. Repositorio

1.1. Repositorio es la carpeta donde se almacenan todos los archivos de un proyecto, junto con el historial de cambios desde el inicio.

1.1.1. Ejemplo:

Repositorio para todas las tareas de la materia base de datos.

“Tareas-BD-27-1”

2. Confirmacion (Commit)

2.1. Commit es cada acción de guardar una modificación del archivo y que puede ser etiquetado con un mensaje que señala el cambio efectuado.

2.1.1. Ejemplo:

Commit del cambio del archivo que contiene la tarea 1 de base de datos.

“Tarea1-InstalacionSGBD.pdf *Agrega imágenes finales yesterday*”

3. Rama (Branch)

3.1. Las ramas son las distintas líneas de trabajo que un proyecto puede tener sin alterar de manera directa la ruta principal de trabajo.

3.1.1. Ejemplo:

Rama principal

“main”

Rama secundaria

“propuestas de mejora”

4. Fusión (merge)

4.1. Una fusión es el proceso de unir todos los cambios de una rama, dentro de otra.

4.1.1. Ejemplo:

Rama secundaria

Rama principal

“propuestas de mejora” → merge → “main”

5. Conflicto de fusión

- 5.1. Ocurre cuando un cambio se realiza en la misma ubicación por dos ramas distintas al mismo tiempo, en este caso Git no puede decidir qué modificación conservar. Se resuelve manualmente para elegir la versión correcta.

5.1.1. Ejemplo:

Línea original (main)

“Título: El camino a la casa *verde*”

Rama 1

Rama 2

“Título: El camino a la casa *azul*”

Título: El camino a la casa *rosa*”

6. Pull Request

6.1. Es la petición que se realiza al equipo (en Github) cuando se ha concluido con los cambios en una rama y se desea revisión antes de una fusión con otra.

6.1.1. Ejemplo:

```
Base(destino) ← Compare (origen)
```

```
Base(main) ← Compare (imagenes-extras)
```

Título:

Agrega imágenes extra a la Practica1

Descripción:

Equipo en este Pull Request se implementa

-Todas las imágenes faltantes en formato PNG.

-Pie de foto para cada uno de los agregados.

Se pide revisión para su posterior adición (merge) a la rama principal.

7. Archivo .gitignore

7.1. El archivo .gitignore, sirve para indicarle a Github qué archivos o carpetas no debe rastrear y mejor ignorar.

7.1.1. Ejemplo:

Archivo → .gitignore

Dentro → contraseñas-BaseDeDatos.txt

8. Archivo READ ME

8.1. Es el archivo de presentación de cada repositorio, en este se agrega todo el texto que explique el proyecto, depende del contenido del repositorio, vienen diversas indicaciones en este archivo READ ME.

8.1.1. Ejemplo:

Brenda Roa Venegas

Número de boleta

Inteligencia Artificial

** Índice del contenido completo de la Práctica 1

FLUJO DE TRABAJO EN RAMAS Y REVISIÓN PREVIO A FUSIÓN

(Feature Branch Workflow)

El flujo de trabajo basado en ramas consiste en desarrollar nuevas funcionalidades, corregir errores o experimentar con ideas dentro de una línea de trabajo aislada en lugar de hacerlo sobre la línea principal. Esto permite trabajar con total libertad sin alterar el código que ya está funcionando, protegiendo así la rama principal contra fallos por cambios accidentales.

Además de facilitar el desarrollo independiente, este flujo aprovecha las solicitudes de extracción (*pull requests*) para abrir un canal de revisión y debate sobre los cambios propuestos. De esta manera, el código se revisa entre pares antes de fusionarse, lo que brinda al equipo la oportunidad de validar la solución, detectar errores a tiempo, garantizar la calidad y mantener estándares compartidos en todo el proyecto.

CUENTA EN GITHUB Y REPOSITORIO

Dirección del repositorio:

<https://github.com/oatbrend/PRACTICA1-DB>

Captura de la salida “*git log --oneline --graph --all*”:

```
bren@Brens-MacBook-Air practica1-db % git log --oneline --graph --all
* 41f7f0b (HEAD -> main, origin/main) Merge pull request #1 from oatbrend/feature/Ejercicio-4
|\
| * a9967ea (origin/feature/Ejercicio-4, feature/Ejercicio-4) Implementa Artículo 1
|/
* d91959e Modifica evidencia entregable
* 6f178cd Agrega Ejercicio1 Practico
* 68938b8 Agrega Ejercicio2 Practico
* cbec65a Agrega Ejercicio 2 parte A
* 777fc8e Modifica Indice de Practica1
* 161dc00 Agrega Ejercicio 1 parte A
* 1d89677 Agrega archivo .gitignore omite archivos temp
* 0e60450 Agrega archivo README con datos personales e Indice de practica
bren@Brens-MacBook-Air practica1-db %
```

Captura del Pull Request fusionado:

The screenshot shows a GitHub interface for a repository named 'oatbrend / PRACTICA1-DB'. The 'Pull requests' tab is selected, showing a pull request titled 'Implementa artículo 1 #2'. The pull request is marked as 'Merged' and shows it was merged 1 commit into the 'main' branch from the 'Ejercicio-4' branch. Below the title, there are statistics: 0 Conversations, 1 Commit, 0 Checks, and 1 File changed. A comment from the owner 'oatbrend' is visible, stating: 'Descripcion: En este Pull Request se implementa: -El primer Artículo Científico relacionado con el uso de las bases de datos.' Below the comment, a commit graph shows the commit 'Implementa artículo 1' (hash 03e75c5) being merged into the 'main' branch. A message at the bottom states 'Pull request successfully merged and closed' and suggests deleting the 'Ejercicio-4' branch. The right sidebar shows a list of repository items with their status (e.g., 'Rev', 'No r', 'Still', 'Ass', 'No c', 'Lab', 'Non', 'Proj', 'Non', 'Mile', 'No r', 'Dev', 'Suc', 'issu').

REFERENCIAS

- Microsoft. (2025). ¿Qué es el control de versiones? Microsoft Learn.
<https://learn.microsoft.com/es-es/devops/develop/git/what-is-version-control>
- Microsoft. (2025). ¿Qué es Git? Microsoft Learn.
<https://learn.microsoft.com/es-es/devops/develop/git/what-is-git>
- Unity. Sistema de control de versiones. Glosario de Unity.
<http://unity.com/es/glossary/version-control>
- Atlassian. Software de control de versiones. Bitbucket.
<https://bitbucket.org/product/es/version-control-software>
- Silveira, P. (2021). Git y GitHub: Qué son y primeros pasos. Blog de Alura Cursos.
<https://www.aluracursos.com/blog/git-y-github-que-son-y-primeros-pasos>
- Kinsta. (2025). *Git vs GitHub: ¿Cuál es la diferencia y cómo empezar?* Kinsta Blog.
<https://kinsta.com/es/blog/git-vs-github/>
- GitHub. *¿Qué es GitHub?* Documentación de GitHub.
<https://docs.github.com/es/get-started/start-your-journey/what-is-github>
- GitHub. *Acerca de las ramas.* Documentación de GitHub.
<https://docs.github.com/es/pull-requests/reference/branches>
- Atlassian. *Flujo de trabajo Feature Branch en Git.* Atlassian Git Tutorials.
<https://www.atlassian.com/git/tutorials/comparing-workflows/feature-branch-workflow>
- Mergify. (2025). *Feature branch workflow: A practical guide for Git.* Mergify Blog.
<https://mergify.com/blog/feature-branch-workflow-a-practical-guide-for-git>